ACTIVIDAD PARCIAL 2 · SECCIÓN 4

BFS y DFS

Búsqueda ciega en un laberinto y en el 8-puzzle

2.4 Algoritmos de búsqueda ciega (BFS / DFS)

UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

ALUMNA
Erandy Naomy Rosas Toriz

GRUPO
171 · Cuatrimestre 7

DOCENTE
Mtro. Javier García Lara

ASIGNATURA
Fundamentos de Inteligencia Artificial

PERIODO
12 al 26 de octubre de 2026

ENTREGA
25 de octubre de 2026 · Individual

Repositorio:

1 PLANTEAMIENTO DEL PROBLEMA

La **búsqueda ciega** (o no informada) explora un espacio de estados sin ninguna pista sobre qué tan cerca está la meta. Solo conoce los estados, las acciones y si un estado es la meta. Los dos algoritmos básicos son la **búsqueda en anchura (BFS)**, que explora nivel por nivel, y la **búsqueda en profundidad (DFS)**, que sigue un camino hasta el fondo antes de retroceder (Russell & Norvig, 2020).

Se aplican a dos problemas: encontrar la salida de un **laberinto** en cuadrícula y resolver un **8-puzzle**. Se comparan tiempo, nodos expandidos, longitud del camino y memoria.

PREGUNTA GUÍA
¿Qué camino encuentra cada algoritmo, cuánto trabajo y memoria necesita, y para qué tipo de problema conviene cada uno?

▸ Objetivos

- Representar los dos problemas como grafos de estados.
- Implementar BFS y DFS en Python con un conjunto de visitados.
- Comparar los resultados, analizar su complejidad temporal y espacial y concluir sobre su idoneidad.

2 REPRESENTACIÓN DEL GRAFO

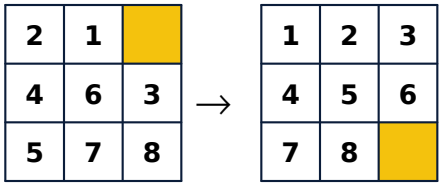
Los grafos se representan de forma **implícita**: no se guarda una lista de adyacencia completa. Cada problema ofrece una función *vecinos(estado)* que genera los sucesores, y los algoritmos construyen el grafo conforme lo exploran. Cada estado es un nodo y cada acción permitida es una arista.

▸ Problema 1 · Laberinto

Elemento	Formulación
Nodo (estado)	Celda libre (fila, columna) de una matriz de 41 × 41; 1 = pared, 0 = pasillo. Hay **853** celdas libres (nodos).
Estado inicial / meta	(1, 1) esquina superior izquierda / (39, 39) esquina inferior derecha.
Aristas (acciones)	Arriba, derecha, abajo, izquierda, si la celda destino no es pared. El grafo es no dirigido.
Costo de paso	1 por movimiento.
Generación	Laberinto de 20 × 20 celdas tallado con DFS aleatorio (semilla 3) y con 10 % de paredes derribadas para crear ciclos, de modo que existan varios caminos.

▸ Problema 2 · 8-puzzle

Elemento	Formulación
Nodo (estado)	Acomodo de las 8 fichas y el hueco en el tablero de 3 × 3 (tupla de 9 números; 0 = hueco). Hay 9!/2 = **181,440** estados alcanzables.
Aristas (acciones)	Mover el hueco arriba, derecha, abajo o izquierda (2 a 4 acciones según su posición).
Costo de paso	1 por movimiento de ficha.
Estado inicial	Generado con 22 movimientos aleatorios desde la meta (siempre resoluble); se muestra a la izquierda.
Meta	(1, 2, 3, 4, 5, 6, 7, 8, 0); se muestra a la derecha.



3 ALGORITMOS Y COMPLEJIDAD

```
BFS (cola FIFO)                          DFS (pila LIFO)
frontera <- cola [inicio]                frontera <- pila [(inicio, None)]
visitados <- {inicio}                    visitados <- {}
mientras frontera no vacia:              mientras frontera no vacia:
    u <- sacar el primero                    u <- sacar el ultimo
    si u es meta: devolver camino            si u en visitados: continuar
    para cada vecino v de u:                 marcar u como visitado
        si v no esta en visitados:           si u es meta: devolver camino
            marcar v ; encolar v             para cada vecino v de u:
devolver fracaso                                 si v no esta visitado: apilar v
```

Pseudocódigo de BFS y DFS con conjunto de visitados (código completo en el Anexo A).

Sea **b** el factor de ramificación, **d** la profundidad de la meta más cercana, **m** la profundidad máxima del espacio, y **V** y **E** los nodos y aristas del grafo.

Propiedad	BFS	DFS
Tiempo (búsqueda en árbol)	$O(b^d)$	$O(b^m)$
Espacio (búsqueda en árbol)	$O(b^d)$	$O(b \cdot m)$
Tiempo (grafo con visitados)	$O(V + E)$	$O(V + E)$
Espacio (grafo con visitados)	$O(V)$	$O(V)$
Completo	Sí	Sí en espacios finitos con visitados
Óptimo (costo 1 por paso)	Sí: camino con menos pasos	No: devuelve el primero que halla

Tabla 1. Complejidad y propiedades. Este trabajo usa búsqueda en grafo (con visitados), por lo que ambos guardan hasta O(V) estados.

4 SOLUCIÓN Y RESULTADOS

4.1 Laberinto de 20 × 20 celdas

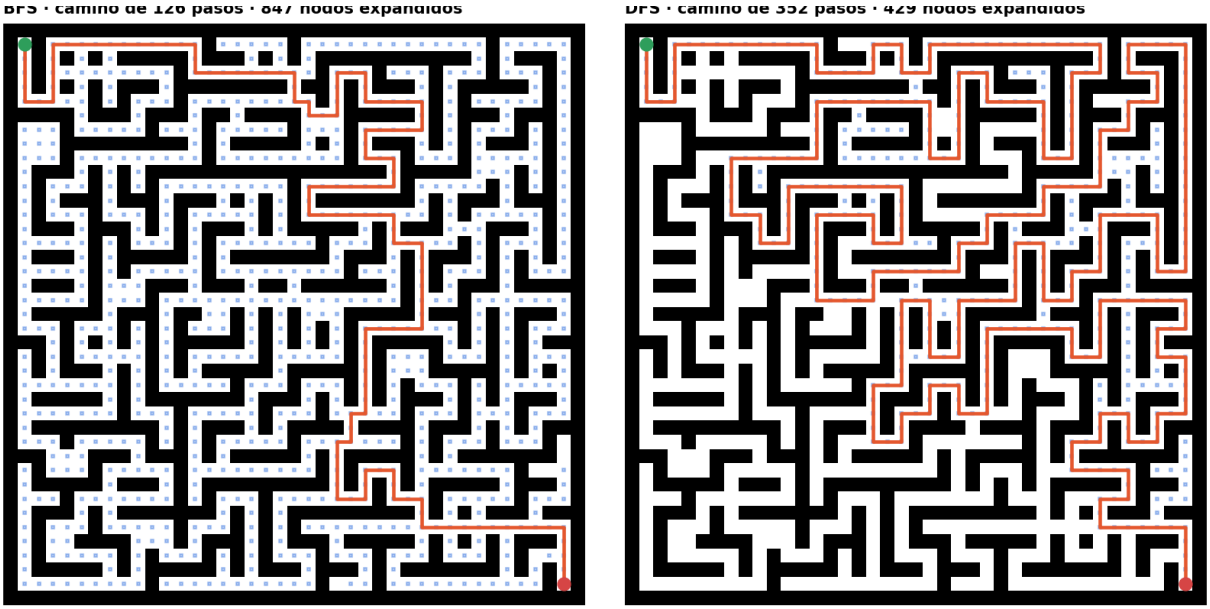


Figura 1. Camino solución (naranja) y celdas expandidas (azul) con BFS y con DFS en el mismo laberinto.

Algoritmo	Pasos del camino	Nodos expandidos	Frontera máxima	Tiempo (ms)
BFS	126	847	23	1.33
DFS	352	429	57	0.73

Tabla 2. Resultados en el laberinto (853 celdas libres).

4.2 8-puzzle

Algoritmo	Movimientos de la solución	Nodos expandidos	Frontera máxima	Tiempo (ms)
BFS	18	20,451	10,017	44.5
DFS	26,722	26,945	20,853	117.2

Tabla 3. Resultados en el 8-puzzle. BFS halló la solución óptima de 18 movimientos; DFS encontró una de 26,722.

4.3 Escalamiento con el tamaño del laberinto

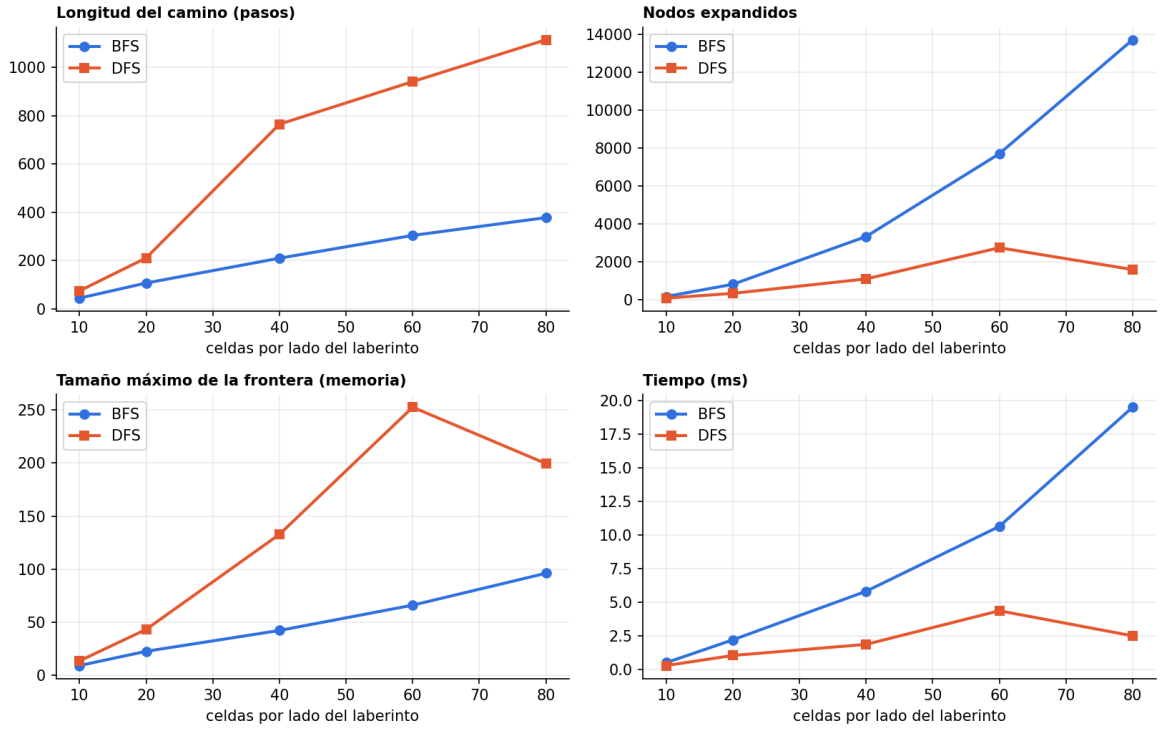


Figura 2. Promedio de 5 laberintos por tamaño.

Celdas	Pasos BFS	Pasos DFS	Exp. BFS	Exp. DFS	Frontera BFS	Frontera DFS
10	45	75	187	102	9	14
20	108	212	828	352	23	43
40	210	765	3,342	1,113	42	133
60	305	942	7,706	2,754	66	252
80	378	1115	13,709	1,605	96	199

Tabla 4. Valores promedio por tamaño (celdas por lado). Exp. = nodos expandidos.

5 COMPARACIÓN E IDONEIDAD

- **Calidad del camino.** BFS siempre devuelve el camino más corto. En el laberinto DFS dio 352 pasos contra 126 (2.8 veces más); en el 8-puzzle, 26,722 movimientos contra 18.
- **Nodos expandidos.** En el laberinto DFS expandió menos (429 contra 847), porque avanza directo en vez de revisar todo un nivel antes de pasar al siguiente; en el 8-puzzle ocurrió lo contrario (26,945 contra 20,451). El resultado de DFS depende de qué tan pronto toma el camino correcto, por eso varía entre laberintos (Tabla 4).
- **Memoria.** La teoría dice que DFS necesita $O(b \cdot m)$ frente a $O(b^d)$ de BFS, pero eso aplica a búsqueda en árbol. Con conjunto de visitados ambos llegan a $O(V)$, y aquí la frontera de DFS fue incluso mayor (57 contra 23 en el laberinto; 20,853 contra 10,017 en el 8-puzzle) porque la pila acumula vecinos pendientes.

- **Tiempo.** Los tiempos son de milisegundos y dependen de la computadora; la comparación fiable es la de nodos expandidos.

Situación	Algoritmo recomendado
Se necesita el camino con menos pasos (costo uniforme por paso)	**BFS**
La meta está cerca del inicio	**BFS**: la encuentra en pocos niveles
Basta con encontrar cualquier solución, o el espacio es muy profundo	**DFS**: avanza rápido hacia el fondo
Recorrer todo el grafo (componentes, ciclos, retroceso o backtracking)	**DFS**
Los costos de las acciones son distintos	Ninguno de los dos: usar Dijkstra o A* (S5)

Tabla 5. Idoneidad según el tipo de problema.

6 CONCLUSIONES REFLEXIVAS

- Con BFS la calidad de la solución está garantizada, pero hay que explorar mucho espacio. Con DFS el trabajo puede ser menor, pero el camino puede ser muy largo.
- El 8-puzzle muestra el límite de la búsqueda ciega: aun con 181,440 estados, BFS expandió más de 20 mil. En problemas más grandes (15-puzzle, mapas reales) se necesitan heurísticas, que es el tema de la S5.
- Una afirmación de teoría (DFS usa menos memoria) puede no cumplirse al implementar la búsqueda en grafo con visitados; conviene medir en lugar de suponer.
- Limitaciones: se usó un solo estado inicial del 8-puzzle, un laberinto base y 5 laberintos por tamaño; los números exactos cambian con otras semillas, pero las tendencias (camino más corto de BFS, camino largo de DFS) se mantuvieron.

7 DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Herramienta	Claude (Anthropic, 2026), modelo Claude Sonnet 5.5, con ejecución de código Python.
Prompt (resumen)	Se entregó la hoja de la Actividad Parcial 2 y se pidió el código y el PDF de la sección 4 (2.4 Algoritmos de búsqueda ciega, BFS y DFS).
Uso	Diseño de los problemas y experimentos, escritura del código (BFS, DFS, generador de laberintos, 8-puzzle, gráficas), borrador del texto y maquetación del PDF (reportlab).
Verificación	El código se ejecutó completo y se revisó que ambos algoritmos llegaran a la meta. La solución de BFS en el 8-puzzle se contrastó con la condición de óptimo por niveles. Las referencias deben cotejarse con las fuentes originales antes de entregar.

R REFERENCIAS (APA 7.ª ED.)

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Russell, S., & Norvig, P. (2020). *Inteligencia Artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing, 1*(2), 146-160. https://doi.org/10.1137/0201010

A ANEXO A. CÓDIGO PYTHON

Archivo: s4_bfs_dfs.py

```
"""S4 · Búsqueda ciega: BFS (anchura) y DFS (profundidad) en un laberinto y en el 8-puzzle (2.4)

Se resuelven dos problemas con los mismos algoritmos y se comparan tiempo, nodos expandidos,
longitud del camino y tamaño máximo de la frontera (memoria):
  1) Laberinto en cuadrícula con ciclos (estado = celda; 4 acciones: arriba, derecha, abajo, izquierda)
  2) 8-puzzle (estado = acomodo de las fichas; la acción mueve el hueco)

Representación del grafo: implícita. Cada problema define vecinos(estado) y los algoritmos
generan el grafo conforme lo exploran. Ambos algoritmos usan un conjunto de visitados.

Ejecutar: python s4_bfs_dfs.py
Requiere: pip install numpy matplotlib
Genera:   figuras/laberinto_bfs_dfs.png, figuras/comparacion_tamanos.png y resultados_s4.json
"""
import json
import os
import random
import time
from collections import deque

import matplotlib
matplotlib.use("Agg")  # genera PNG sin abrir ventana
import matplotlib.pyplot as plt
import numpy as np

AQUI = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(AQUI, "figuras")
os.makedirs(FIG, exist_ok=True)


# ======================================================================
# 1. Algoritmos de búsqueda ciega (genéricos: sirven para cualquier problema)
# ======================================================================
def reconstruir(padre, meta):
    camino = [meta]
    while padre[camino[-1]] is not None:
        camino.append(padre[camino[-1]])
    return camino[::-1]


def bfs(inicio, es_meta, vecinos):
    """Búsqueda en anchura: frontera = COLA (FIFO). Encuentra el camino con menos pasos."""
    t0 = time.perf_counter()
    padre = {inicio: None}                # sirve también como conjunto de visitados
    frontera = deque([inicio])
    expandidos, max_frontera, orden = 0, 1, []
    while frontera:
        u = frontera.popleft()
        expandidos += 1
        orden.append(u)
        if es_meta(u):
            break
        for v in vecinos(u):
            if v not in padre:
                padre[v] = u
                frontera.append(v)
        max_frontera = max(max_frontera, len(frontera))
    else:
        return None
    camino = reconstruir(padre, u)
    return {"camino": camino, "pasos": len(camino) - 1, "expandidos": expandidos, "max_frontera": max_frontera,
            "tiempo_ms": (time.perf_counter() - t0) * 1000, "orden": orden}


def dfs(inicio, es_meta, vecinos):
    """Búsqueda en profundidad: frontera = PILA (LIFO). Encuentra un camino, no necesariamente el más corto."""
    t0 = time.perf_counter()
    padre, pila = {}, [(inicio, None)]
    expandidos, max_frontera, orden = 0, 1, []
    while pila:
        u, p = pila.pop()
        if u in padre:
            continue
```

```
        padre[u] = p
        expandidos += 1
        orden.append(u)
        if es_meta(u):
            break
        for v in reversed(vecinos(u)):     # reversed: se explora primero el primer vecino
            if v not in padre:
                pila.append((v, u))
        max_frontera = max(max_frontera, len(pila))
    else:
        return None
    camino = reconstruir(padre, u)
    return {"camino": camino, "pasos": len(camino) - 1, "expandidos": expandidos, "max_frontera": max_frontera,
            "tiempo_ms": (time.perf_counter() - t0) * 1000, "orden": orden}


# ============================================================================
# 2. Problema 1: laberinto
# ============================================================================
def generar_laberinto(celdas, semilla, ciclos=0.10):
    """Laberinto de celdas x celdas. 1 = pared, 0 = pasillo. Se talla con DFS aleatorio y luego se
    derriban algunas paredes ('ciclos') para que existan varios caminos posibles."""
    rng = random.Random(semilla)
    n = 2 * celdas + 1
    g = np.ones((n, n), dtype=int)
    pila = [(1, 1)]
    g[1, 1] = 0
    while pila:
        r, c = pila[-1]
        vec = [(r + dr, c + dc, dr, dc) for dr, dc in ((-2, 0), (2, 0), (0, -2), (0, 2))
               if 0 < r + dr < n - 1 and 0 < c + dc < n - 1 and g[r + dr, c + dc] == 1]
        if vec:
            nr, nc, dr, dc = rng.choice(vec)
            g[r + dr // 2, c + dc // 2] = 0
            g[nr, nc] = 0
            pila.append((nr, nc))
        else:
            pila.pop()
    paredes = [(r, c) for r in range(1, n - 1) for c in range(1, n - 1)
               if g[r, c] == 1 and ((g[r - 1, c] == 0 and g[r + 1, c] == 0) or (g[r, c - 1] == 0 and g[r, c + 1] == 0))]
    for r, c in rng.sample(paredes, int(len(paredes) * ciclos)):
        g[r, c] = 0
    return g


def problema_laberinto(g):
    n = g.shape[0]
    inicio, meta = (1, 1), (n - 2, n - 2)

    def vecinos(s):
        r, c = s
        return [(r + dr, c + dc) for dr, dc in ((-1, 0), (0, 1), (1, 0), (0, -1)) if g[r + dr, c + dc] == 0]
    return inicio, (lambda s: s == meta), vecinos


# ============================================================================
# 3. Problema 2: 8-puzzle
# ============================================================================
META_8 = (1, 2, 3, 4, 5, 6, 7, 8, 0)


def vecinos_8(s):
    h = s.index(0)
    r, c = divmod(h, 3)
    res = []
    for dr, dc in ((-1, 0), (0, 1), (1, 0), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            l = list(s)
            j = nr * 3 + nc
            l[h], l[j] = l[j], l[h]
            res.append(tuple(l))
    return res


def revolver_8(pasos, semilla):
    """Parte de la meta y aplica movimientos aleatorios (sin deshacer el anterior): estado siempre resoluble."""
    rng = random.Random(semilla)
    s, anterior = META_8, None
    for _ in range(pasos):
```

```
        opciones = [v for v in vecinos_8(s) if v != anterior]
        anterior, s = s, rng.choice(opciones)
    return s


# ============================================================================
# 4. Experimentos
# ============================================================================
def resumen(r):
    return {k: r[k] for k in ("pasos", "expandidos", "max_frontera", "tiempo_ms")}


def comparar_tamanos(tamanos=(10, 20, 40, 60, 80), semillas=range(5)):
    filas = []
    for celdas in tamanos:
        acum = {"bfs": [], "dfs": []}
        for s in semillas:
            g = generar_laberinto(celdas, s)
            ini, meta, vec = problema_laberinto(g)
            acum["bfs"].append(resumen(bfs(ini, meta, vec)))
            acum["dfs"].append(resumen(dfs(ini, meta, vec)))
        fila = {"celdas": celdas}
        for alg, lst in acum.items():
            for k in lst[0]:
                fila[f"{alg}_{k}"] = float(np.mean([x[k] for x in lst]))
        filas.append(fila)
    return filas


# ============================================================================
# 5. Gráficas
# ============================================================================
NAVY, AMARILLO, AZUL, GRIS = "#0B1D3A", "#F4C20D", "#2F6FDE", "#D5DBE5"


def dibujar_laberinto(ax, g, r, titulo):
    img = np.where(g == 1, 0.0, 1.0)
    ax.imshow(img, cmap="gray", vmin=0, vmax=1)
    ex = np.array(r["orden"])
    ax.scatter(ex[:, 1], ex[:, 0], s=4, color=AZUL, alpha=0.35, marker="s")
    cam = np.array(r["camino"])
    ax.plot(cam[:, 1], cam[:, 0], color="#E4572E", lw=2.2)
    n = g.shape[0]
    ax.scatter([1], [1], s=60, color="#2E9E5B", zorder=5); ax.scatter([n - 2], [n - 2], s=60, color="#D64545", zorder=5)
    ax.set_title(titulo, fontsize=10, fontweight="bold", loc="left"); ax.axis("off")


def graficas(g, rb, rd, filas):
    fig, ax = plt.subplots(1, 2, figsize=(11, 5.6))
    dibujar_laberinto(ax[0], g, rb, f"BFS · camino de {rb['pasos']} pasos · {rb['expandidos']} nodos expandidos")
    dibujar_laberinto(ax[1], g, rd, f"DFS · camino de {rd['pasos']} pasos · {rd['expandidos']} nodos expandidos")
    fig.text(0.5, 0.02, "Azul = celdas expandidas · naranja = camino solución · verde = inicio · rojo = meta", ha="center", fontsi
    fig.tight_layout(rect=[0, 0.04, 1, 1]); fig.savefig(os.path.join(FIG, "laberinto_bfs_dfs.png"), dpi=150); plt.close(fig)

    fig, ax = plt.subplots(2, 2, figsize=(11, 7))
    x = [f["celdas"] for f in filas]
    for a, (k, tit) in zip(ax.flat, [("pasos", "Longitud del camino (pasos)"), ("expandidos", "Nodos expandidos"),
                                     ("max_frontera", "Tamaño máximo de la frontera (memoria)"), ("tiempo_ms", "Tiempo (ms)")]):
        a.plot(x, [f[f"bfs_{k}"] for f in filas], "o-", color=AZUL, lw=2, label="BFS")
        a.plot(x, [f[f"dfs_{k}"] for f in filas], "s-", color="#E4572E", lw=2, label="DFS")
        a.set_title(tit, fontsize=10, fontweight="bold", loc="left"); a.set_xlabel("celdas por lado del laberinto"); a.legend()
        a.spines[["top", "right"]].set_visible(False); a.grid(alpha=0.25)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, "comparacion_tamanos.png"), dpi=150); plt.close(fig)


def main():
    # ----- Laberinto de 20 x 20 celdas -----
    g = generar_laberinto(20, semilla=3)
    ini, meta, vec = problema_laberinto(g)
    rb, rd = bfs(ini, meta, vec), dfs(ini, meta, vec)
    libres = int((g == 0).sum())
    print(f"=== Laberinto 20x20 celdas (matriz {g.shape[0]}x{g.shape[1]}, {libres} celdas libres) ===")
    print(f"{'':6}{'Pasos':>8}{'Expandidos':>12}{'Frontera máx.':>15}{'Tiempo (ms)':>13}")
    for nombre, r in (("BFS", rb), ("DFS", rd)):
        print(f"{nombre:6}{r['pasos']:>8}{r['expandidos']:>12}{r['max_frontera']:>15}{r['tiempo_ms']:>13.3f}")

    # ----- 8-puzzle -----
    inicio8 = revolver_8(22, semilla=5)
    r8b = bfs(inicio8, lambda s: s == META_8, vecinos_8)
    r8d = dfs(inicio8, lambda s: s == META_8, vecinos_8)
```

```
    print(f"\n=== 8-puzzle · inicio {inicio8} ===")
    for nombre, r in (("BFS", r8b), ("DFS", r8d)):
        print(f"{nombre:6}{r['pasos']:>8}{r['expandidos']:>12}{r['max_frontera']:>15}{r['tiempo_ms']:>13.1f}")
    print("Solución óptima (BFS):", r8b["pasos"], "movimientos; el espacio completo tiene 181,440 estados alcanzables")

    # ----- Escalamiento -----
    filas = comparar_tamanos()
    print("\n=== Escalamiento (promedio de 5 laberintos por tamaño) ===")
    print(f"{'celdas':>7}{'BFS pasos':>11}{'DFS pasos':>11}{'BFS exp.':>10}{'DFS exp.':>10}{'BFS front.':>12}{'DFS front.':>12}")
    for f in filas:
        print(f"{f['celdas']:>7}{f['bfs_pasos']:>11.0f}{f['dfs_pasos']:>11.0f}{f['bfs_expandidos']:>10.0f}{f['dfs_expandidos']:>10
              f"{f['bfs_max_frontera']:>12.0f}{f['dfs_max_frontera']:>12.0f}")

    graficas(g, rb, rd, filas)
    quitar = lambda r: {k: v for k, v in r.items() if k not in ("orden", "camino")}
    with open(os.path.join(AQUI, "resultados_s4.json"), "w", encoding="utf-8") as f:
        json.dump({"laberinto": {"libres": libres, "matriz": g.shape[0], "bfs": quitar(rb), "dfs": quitar(rd)},
                   "puzzle": {"inicio": list(inicio8), "bfs": quitar(r8b), "dfs": quitar(r8d)},
                   "escala": filas}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

B ANEXO B. SALIDA DE LA EJECUCIÓN

```
=== Laberinto 20x20 celdas (matriz 41x41, 853 celdas libres) ===
         Pasos  Expandidos  Frontera máx.  Tiempo (ms)
BFS        126         847             23        1.334
DFS        352         429             57        0.725

=== 8-puzzle · inicio (2, 1, 0, 4, 6, 3, 5, 7, 8) ===
BFS         18       20451          10017         44.5
DFS      26722       26945          20853        117.2
Solución óptima (BFS): 18 movimientos; el espacio completo tiene 181,440 estados alcanzables

=== Escalamiento (promedio de 5 laberintos por tamaño) ===
 celdas  BFS pasos  DFS pasos  BFS exp.  DFS exp.  BFS front.  DFS front.
     10         45         75       187       102           9          14
     20        108        212       828       352          23          43
     40        210        765      3342      1113          42         133
     60        305        942      7706      2754          66         252
     80        378       1115     13709      1605          96         199
```